

AIM PRACTICE



תיעוד ומדריך מחלקות מקיף

ניתוח ארכיטקטורה מפורט, הסבר מנגנונים מורכבים ויצאת דופן, ודוגמאות שימוש בקוד עבור פרויקט אימון הירי (Aim Practice).

תאריך יצירה

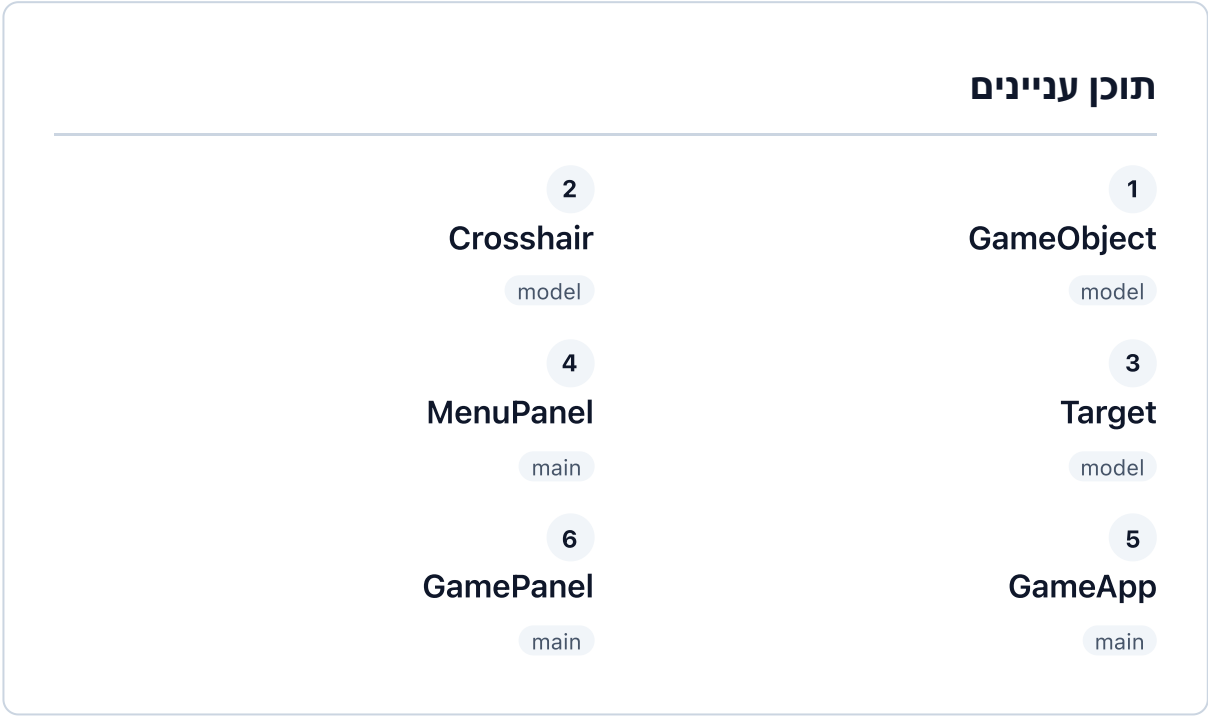
יוני 2026

נוצר על ידי

Antigravity AI (Google DeepMind)

מבנה הפרויקט והמחלקות

פרויקט Aim Practice הוא משחק אימון ירי דו-ממדי (2D Aim Trainer) שנכתב ב-Java Swing. המבנה הארכיטקטוני שלו מבוסס על הפרדה ברורה בין מודלים לוגיים לבין רכיבי תצוגה ומנוע הריצה.



package model

Model

Abstract Class

GameObject

תיאור ותפקיד

מחלקה מופשטת זו מהווה את מחלקת הבסיס לכל העצמים הפיזיים והגרפיים במשחק (הכוונת והמטרה). היא מגדירה את מאפייני היסוד המשותפים של כל עצם במרחב הדו-ממדי: קואורדינטות מיקום, מידות, וצבע. המחלקה מחייבת את כל יורשיה לממש את שיטות הציור והעדכון של העצם, ובכך היא מאפשרת פולימורפיזם וניהול אחיד של מגוון אלמנטים על גבי המסך.

שיטות ומאפייני מפתח

- מאפיינים מוגנים: `protected int x, y` (מיקום הפינה השמאלית-עליונה) ו- `width, height` (ממדי הגוף).
- שיטת `getBounds()`: מחזירה אובייקט `java.awt.Rectangle` המבוסס על מיקום וממדי העצם. משמש כגבול חוסם (Bounding Box) ומאפשר חישובי מגע/התנגשות בסיסיים.
- שיטה מופשטת `update()`: מיועדת לעדכון הלוגיקה הפנימית או הפיזיקה של העצם בכל פריים (למשל, שינוי מיקום לפי קליטת מקשים).
- שיטה מופשטת `draw(Graphics g)`: מיועדת לציור העצם על גבי קנבס הציור של Java AWT.

מה מוזר או מסובך במחלקה?

ייצוג קואורדינטות בפינה השמאלית-עליונה

כמו ברוב ספריות הדו-ממד הגרפיות, קואורדינטות ה-x וה-y של האובייקט אינן מייצגות את מרכזו הגיאומטרי, אלא את **הפינה השמאלית-העליונה** של המלבן החוסם אותו. כאשר נרצה לחשב את המרכז הפיזי של העצם (כמו בחישוב פגיעות ירי או ציור צולב), נצטרך תמיד לבצע התאמה מתמטית: $centerX = x + width / 2$ ו- $centerY = y + height / 2$. בנוסף, למרות שמדובר במחלקה מופשטת שלא ניתן לייצר ממנה מופע באופן ישיר, היא כוללת בנאי (Constructor) מלא, מה שמחייב את כל המחלקות היורשות להשתמש בבנאי של מחלקת האב באמצעות קריאה ל-`super()`.

דוגמת שימוש בקוד

```
// עבור מכשול סטטי במשחק GameObject יצירת מחלקה יורשת מתוך
import model.GameObject;
import java.awt.Color;
import java.awt.Graphics;

public class Wall extends GameObject {

    public Wall(int x, int y, int width, int height) {
        super(x, y, width, height, Color.GRAY); // קריאה לבנאי האב
    }

    @Override
    public void update() {
        // קיר סטטי - אין צורך בעדכון לוגיקה בכל פריים
    }

    @Override
    public void draw(Graphics g) {
        g.setColor(color);
        g.fillRect(x, y, width, height); // ציור מלבן פשוט
    }
}
```

package model

Model Crosshair**תיאור ותפקיד**

מחלקה זו מייצגת את הכוונת של השחקן. הכוונת משמשת ככלי הניווט והירי המרכזי של השחקן על גבי המסך. המחלקה עוקבת אחר מצב המקשים הלחוצים (למעלה, למטה, שמאלה, ימינה במקלדת - חצים או WASD), מעדכנת את מיקומה הפיזי בהתאם למהירות ההתחלתית שלה, ומוודאת שהיא לעולם לא תחרוג מגבולות החלון.

שיטות ומאפייני מפתח

- **בנאי** `Crosshair(int screenWidth, int screenHeight)` : מאתחל את הכוונת בדיוק במרכז המסך, בגודל של 16×16 פיקסלים ובצבע ירוק זוהר.
- שיטות קביעת מצב מקש: `setUpPressed, setDownPressed, setLeftPressed, setRightPressed` . אלו משתמשים במשתנים בוליאניים כדי לעקוב האם המקש לחוץ כעת במקלדת.
- שיטת `update()` : מוזחת את מיקום הפיקסלים (x או y) בהתאם למקשים הלחוצים ומהירות התנועה (`speed = 5`). מיד לאחר מכן מתבצע סינון טווח (clipping) המונע תנועה אל מחוץ למסך.
- שיטת `draw(Graphics g)` : מציגה את הכוונת על ידי ציור קווים דקים המצטלבים במרכז.

מה מוזר או מסובך במחלקה?**חישוב נקודת ההצטלבות ומהירות אלכסונית**

א. ציור ביחס למרכז: בשיטת הציור `draw()`, הכוונת אינה מציירת צלב פשוט בנקודת ה- x, y . היא מחשבת קודם את המרכז הפיזי של מלבן הכוונת: $centerX = x + width / 2$ ו- $centerY = y + height / 2$. שני הקווים המצטלבים נמתחים במדויק דרך נקודה זו (קו אופקי מ- x עד $x+width$ במיקום ה- $centerY$, וקו אנכי מ- y עד $y+height$ במיקום ה- $centerX$).

ב. בעיית המהירות האלכסונית (Vector Magnitude): כאשר השחקן לוחץ על שני מקשים בו-זמנית (למשל, למעלה וימינה), מיקום הכוונת זז ב-5 פיקסלים למעלה וב-5 פיקסלים ימינה באותו פריים. מכיוון שהמהירות מיושמת על שני הצירים בנפרד ללא נרמול וקטורי, השחקן זז באלכסון במהירות של: עבור מהירות ציר 5, נקבל תנועה אלכסונית של כ-7.07 פיקסלים לפריים. תנועה זו מהירה בכ-41% מתנועה ישרה (אופקית או אנכית).

דוגמת שימוש בקוד

```
// יצירת כוונת ועדכון המיקום שלה כתוצאה מאירועי לחיצה  
Crosshair crosshair = new Crosshair(800, 600);  
  
// המשתמש לוחץ על מקש חץ ימינה  
crosshair.setRightPressed(true);  
  
// Game Loop- ירוץ בתוך ה) הרצת עדכון לוגיקה  
crosshair.update();  
  
// זז ימינה ב-5 פיקסלים Crosshair-המיקום של ה  
System.out.println("New X Position: " + crosshair.getX());
```

עמוד 3

תיעוד מחלקות Aim Practice

package model

Model

Target

תיאור ותפקיד

מחלקה זו מייצגת את המטרה הגרפית המופיעה על המסך. המטרה היא עצם סטטי שאינו זז באופן עצמאי אלא משנה את מיקומו באופן אקראי על פני המסך בכל פעם שהוא נפגע על ידי ירייה מוצלחת של השחקן. המחלקה עושה שימוש ביכולות רישום מתקדמות בדו-ממד להפקת מטרה בעיצוב הייטק מרשים.

שיטות ומאפייני מפתח

- שיטת `respawn()`: מחשבת קואורדינטות אקראיות חדשות (X, Y) בטווח המסך, תוך שמירה על שולי ביטחון מובנים של 50 פיקסלים מקצוות המסך כדי למנוע את היווצרות המטרה מחוץ לשדה הראייה הנוח.
- שיטת `draw(Graphics g)`: ממירה את ה-`Graphics` ל-`Graphics2D` ומציירת מטרה מפורטת המורכבת מרקע כתום שקוף למחצה, רשת קווים פנימית, מסגרת מוצקה וגרעין פנימי קטן וזוהר.

מה מוזר או מסובך במחלקה?

נוסחת השוליים וטכניקות הציר המתקדמות

א. נוסחת מניעת ההיווצרות בשוליים: בשיטה `respawn()` () מוגדר שולי ביטחון `margin =` 50. כדי למנוע מהמטרה לצאת מהגבולות הפיזיים הימני והתחתון, הנוסחה מנכה מהקצה של החלון גם את רוחב/גובה המטרה עצמה `(width)`:

$$x = \text{margin} + \text{random.nextInt}(\text{screenWidth} - 2 * \text{margin} - \text{width})$$
 זה מבטיח שהמטרה תוגרל במדויק בטווח המבוקש.

ב. שימוש ב-`Graphics2D` מתקדם: השיטה `draw()` () משתמשת בהמרה ל-`Graphics2D` ובפונקציות מתקדמות: `RenderingHints`: מופעל מנגנון החלקת קצוות (Anti-aliasing) למניעת קצוות משוננים של המלבן המעוגל. `Alpha Transparency`: הרקע של המטרה מצויר בגוון כתום שקוף-למחצה (ערך אלפא של 150 מתוך 255): `new Color(255, 140, 0, 150)`. - קווי רשת פנימיים (`Gridlines`): מצוירים 3 קווים אנכיים ו-3 אופקיים המבוססים על חלוקת מלבן המטרה ל-4 חלקים שווים `(spacing = width / 4)`. - גרעין מרכזי (`Center Core`): מחושב מיקום מרכזי מדויק בריבוע שגודלו חמישית מגודל המטרה.

דוגמת שימוש בקוד

```
// 800x600 יצירת מטרה בחלון בגודל  
Target target = new Target(800, 600);  
  
// הדפסת המיקום הראשוני שהוגרל בבנאי  
System.out.println("X: " + target.getX() + ", Y: " + target.getY());  
  
// העברת המטרה למיקום אקראי חדש (למשל, לאתר פגיעה)  
target.respawn();
```

עמוד 4

תיעוד מחלקות Aim Practice

package main

GUI Panel

MenuPanel

תיאור ותפקיד

מחלקה זו יורשת מ-JPanel ומייצגת את מסך הבית והוראות המשחק. היא מציגה כותרת גדולה ומרשימה בצבע טורקיז זוהר, הוראות מפורטות לשימוש במקלדת ובעכבר, וכפתור מרכזי להתחלת המשחק ("START GAME"). מסך זה נועד להיות עצמאי ומעביר את האישור להתחלת המשחק לפנל הראשי באמצעות Callback (מנגנון קריאה חוזרת).

שיטות ומאפייני מפתח

- **בנאי MenuPanel(Runnable onStartCallback):** מקבל מופע של Runnable שיש להפעיל ברגע שלחיצת הכפתור מתרחשת. הבנאי מארגן את הרכיבים השונים באמצעות שילוב מנהלי פריסה (Layout Managers).
- **שימוש ב-BoxLayout:** הפאנל הפנימי מסודר אנכית (Y_AXIS), ומאפשר סידור נוח של כותרת, הוראות טקסט וכפתור ההפעלה בזה אחר זה.
- **שיטת טיפול באירועים:** מאזין לפעולת הכפתור (ActionListener) מפעיל את הקולבק onStartCallback.run() אם הוא אינו ריק.

מה מוזר או מסובך במחלקה?

תצורת תיבת ההוראות (JTextArea) ופריסה דינמית

א. ביטול אפקטים של JTextArea: כדי להציג הוראות מעוצבות הכוללות ירידות שורה, נעשה שימוש ברכיב JTextArea. אולם, כברירת מחדל רכיב זה מיועד לעריכה ויש לו רקע לבן ומסגרת. כדי לגרום לו להתמזג בעיצוב, הוגדרו עבורו הפרמטרים הבאים: -
`instructionsArea.setOpaque(false);` מונע ציור רקע לבן, ובכך חושף את הצבע השחור של הפאנל הראשי. -
`instructionsArea.setEditable(false);` הופך אותו לטקסט לקריאה בלבד. -
`instructionsArea.setFocusable(false);` מונע מהמשתמש לבחור או להציב את סמן הכתיבה (Caret) בתוך התיבה, ובכך שומר על מקשי המקלדת זמינים למשחק.

ב. פריסה דינמית וריווחים: נעשה שימוש ב-`Box.createVerticalGlue()` (בראש ובתחתית הרכיבים. מטרתו של ה-Glue היא להתנהג כקפיץ אלסטי שדוחף את כל הרכיבים (כותרת, הוראות, כפתור) יחד למרכז האנכי של החלון. הריווחים הקבועים מבוצעים בעזרת `Box.createRigidArea(new Dimension(0, 30))`.

דוגמת שימוש בקוד

```
// מאתחיל את תפריט המשחק ומספק קולבק שמדפיס הודעה  
MenuPanel menuPanel = new MenuPanel(() -> {  
    System.out.println("קולבק הופעל!");  
});
```

עמוד 5

תיעוד מחלקות Aim Practice

package main

App Entry

GameApp

| תיאור ותפקיד

מחלקה זו מהווה את שער הכניסה הראשי לתוכנית (Entry Point). היא מכילה את מתודת ה-main ומגדירה את ה-JFrame (חלון האפליקציה). תפקידה המרכזי הוא לנהל את המבנה הראשי של החלונות ולהגדיר את החוקים למעבר בין מסך התפריט (MenuPanel) למסך המשחק הפעיל (GamePanel) על ידי שימוש במנהל הפריסה CardLayout.

| שיטות ומאפייני מפתח

- שיטת `main(String[] args)`: השיטה הראשונה שמורצת על ידי ה-Java Virtual Machine.
- שימוש ב- `CardLayout`: מנהל פריסה המאפשר להחזיק מספר רכיבים כמו "כרטיסיות" (Cards) ולהציג בכל רגע נתון רק כרטיסייה אחת על גבי החלון הראשי.
- שימוש ב- `JFrame`: יצירת החלון הראשי של מערכת ההפעלה, הגדרת הגודל הקבוע שלו (800x600) ומניעת שינוי גודל החלון באופן ידני (`Resizable: false`).

| מה מוזר או מסובך במחלקה?

SwingUtilities.invokeLater ובטיחות תהליכונים (Thread Safety)

א. **בטיחות תהליכונים ב-Swing**: במתודה `main`, קוד אתחול ה-GUI כולו עטוף בקריאה: `SwingUtilities.invokeLater(() -> { ... });`.
למה זה חובה? ספריות הגרפיקה של Java (Swing ו-AWT) אינן בטוחות לשימוש מרובה תהליכונים. כל שינוי, יצירה או עדכון של רכיבי ממשק משתמש חייבים לרוץ אך ורק על גבי תהליכון מיוחד בשם **Event Dispatch Thread (EDT)**. קריאה ל-`invokeLater` מכניסה את קוד אתחול החלון לתור המשימות של ה-EDT, מה שמונע שגיאות סנכרון לא צפויות ומצבי מרוץ (Race Conditions) מול ה-Thread הראשי של ה-JVM.

ב. **העברת פוקוס בעת החלפת כרטיס (Card Routing)**: כאשר השחקן לוחץ על כפתור תחילת המשחק, ה-Callback מחליף את התצוגה לכרטיס המשחק ("GAME"). אולם, באותו רגע המקלדת עלולה לאבד את המיקוד שלה. כדי לפתור זאת, מופעלות שתי פקודות רצופות: `gamePanel.requestFocusInWindow()`; - המבקשת להחזיר את פוקוס המקלדת לפאנל המשחק. `gamePanel.startGame()`; - המניעה את הלולאה הפנימית של המשחק.

| דוגמת שימוש בקוד

```
// נקודת הכניסה של היישום כולו
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        JFrame frame = new JFrame("Aim Practice");
        frame.setVisible(true);
    });
}
```

package main

Engine

GUI Panel

GamePanel

תיאור ותפקיד

זוהי המחלקה החשובה והמורכבת ביותר בפרויקט, המהווה את מנוע המשחק (Game Engine) בפועל. היא יורשת מ-JPanel ומממשת את ממשק ה-Runnable. תפקידה כוללים ניהול מחזור חיי תהליכון המשחק (FPS Game Loop 60), קליטת קלט משולב (מקלדת ועכבר), ניהול מצבי המשחק (פעיל, מושהה, סיום), חישוב פגיעות ירי מבוסס מרחק, וציור כלל האלמנטים על המסך - כולל זרועות מגוף ראשון עם אפקט הדף (Recoil) ותנועה תלת-ממדית מדומה (Pseudo-3D parallax).

שיטות ומאפייני מפתח

- שיטת `startGame()`: יוצרת ומפעילה תהליכון (Thread) חדש המריץ את המשחק ברקע באופן עצמאי.
- שיטת `run()`: הלולאה הראשית של המשחק. מנהלת את קצב הריצה בעזרת השוואת זמנים מדויקת.
- שיטת `shoot()`: מופעלת בעת לחיצה על רווח או קליק שמאלי. היא מגדילה את מונה היריות, מפעילה אפקט רתע ומשווה מרחק למטרה.
- שיטת `drawFirstPersonArms(Graphics g)`: מציירת נשק וידיים בדו-ממד מלא, המשנים את מיקומם דינמית בהתאם למיקום הכוונת.
- שיטת `drawHUD(Graphics g)`: מציגה לוח נתונים בזמן אמת - דיוק נוכחי (Kills/Shots), קצב הריגות בשנייה (Kills/Sec), שעון עצר וניקוד.

מה מוזר או מסובך במחלקה?

ניתוח מנגנוני המשחק המתקדמים

1. **לולאת FPS קבועה (בשיטה run):** המשחק פועל בקצב קבוע של 60 פריימים בשנייה. בשיטת הלולאה הראשית, נמדד הזמן שלקח לבצע את העדכון והציור בנו-שניות (elapsed). זמן זה מומר למילישניות ומנוכה מזמן הפריים הרצוי (כ-16.6 מילישניות): $wait = targetTime - elapsed$. אם מתקבל ערך חיובי, התהליכון מורדם למשך זמן זה. אם הוא שלילי (מכיוון שהעיבוד לקח יותר מ-16.6 מילישניות), התהליכון אינו מורדם כלל כדי לפצות על העומס.

2. **זיהוי פגיעה מעגלי (בשיטה shoot):** למרות שהמטרה (Target) מוגדרת כמלבן ומצוירת כריבוע עם פינות מעוגלות, חישוב הפגיעה מתבצע כאילו היא **עיגול מושלם**. השיטה מחשבת את המרחק הגיאומטרי בין נקודת מרכז הכוונת לבין מרכז המטרה בעזרת הנוסחה למציאת מרחק בין שתי נקודות (Point2D.distance). ירייה נחשבת לפגיעה אך ורק אם המרחק קטן או שווה לרדיוס המטרה (חצי מרוחב המטרה): $distance \leq target.getWidth() / 2.0$. אם השחקן יורה בפינות הקיצוניות של ריבוע המטרה, הירי לא ייקלט כפגיעה.

3. הדמיית רתע/הדף (Recoil Decelerate): בעת ביצוע ירי, המשתנה recoilOffset מקבל ערך של 20. בשיטת העדכון update(), הערך הזה דועך בהדרגה בכל פריים ב-2.0 יחידות: `if recoilOffset > 0) recoilOffset -= 2.0`; בשיטת ציור הנשק, המיקום האנכי של הנשק מושפע מפרמטר זה: `baseY = h + 20 + offsetY + recoilOffset`. זה גורם לנשק "לקפוץ" מיידית למטה ב-20 פיקסלים בעת הלחיצה, ולהחליק בחזרה למעלה בצורה רציפה וחלקה על פני 10 פריימים.

4. הדמיית תנועה תלת-ממדית מדומה (Pseudo-3D Parallax): הנשק והידיים מצוירים במרכז החלק התחתון. כדי להעניק תחושת עומק ומיקוד במרחב תלת-ממדי, מחושב היסט (Offset) יחסי למיקום הכוונת: `offsetX = (crosshairX - w / 2.0) * 0.3; offsetY = (crosshairY - h / 2.0) * 0.3`; כאשר השחקן מזיז את הכוונת, הנשק והזרועות מופעלים עם היסט זה. מכיוון שהיסט הוא 0.3 (30% בלבד מתנועת הכוונת), נוצרת פרלקסה מרהיבה שבה הכוונת זזה מהר יותר מהנשק, מה שמדמה תנועת מבט ומקנה למשחק מראה ייחודי של משחקי יריות מגוף ראשון קלאסיים (Doom-style).

5. שימוש בשיבוט Graphics (שיטת g.create()): בשיטה `drawFirstPersonArms`, במקום לעבוד ישירות על מופע ה-`Graphics` המקורי (שמשפיע באופן גלובלי על כל שאר הציורים הבאים), מתבצע שיבוט מקומי: `Graphics2D g2d = (Graphics2D) g.create()`; פעולות הזזה (`translate`) ושינוי קנה מידה (`scale`) מבוצעות על גבי המשובט, ובסוף השיטה הוא נמחק בעזרת `g2d.dispose()`; דבר זה מונע שיבוש של מיקומי רכיבים אחרים (כמו ה-HUD שמצוירים לאחר מכן).

דוגמת שימוש בקוד

```
// הפעלת המשחק 600x800 הפאנל בגודל 800
GamePanel gamePanel = new GamePanel(800, 600);
gamePanel.startGame();
```